

list all spring microservices topics that need to be known
by 7-8 year java developer

For a **7–8 year Java/Spring Boot developer**, you should know microservices beyond just creating REST APIs. You're expected to understand **architecture, distributed-system problems, resilience, security, deployment, observability, and design trade-offs**.

Here is a comprehensive roadmap.

1. Microservices Fundamentals

- What are microservices?
 - Monolith vs modular monolith vs microservices
 - Advantages and disadvantages
 - When **not** to use microservices
 - Characteristics of a microservice
 - Independent deployment
 - Independent scaling
 - Loose coupling and high cohesion
 - Bounded context
 - Domain-driven design basics
 - Database per service
 - Shared database problems
 - Synchronous vs asynchronous communication
 - Stateless services
 - 12-factor application principles
 - Conway's Law
 - Distributed systems fundamentals
-

2. Service Decomposition

Very important for senior developers.

- How to break a monolith into microservices
- Decomposition by business capability
- Decomposition by subdomain
- Bounded contexts
- Identifying service boundaries

- Avoiding distributed monoliths
- Avoiding nano-services
- Shared libraries vs shared services
- Service ownership
- Strangler Fig pattern
- Anti-corruption layer
- Branch by Abstraction
- Database decomposition

Interview question:

How would you convert a large monolithic application into microservices?

You should be able to explain this end-to-end.

3. Spring Boot for Microservices

You should know:

- Spring Boot auto-configuration
- Starters
- Dependency management
- Spring Boot Actuator
- Externalized configuration
- Profiles
- application.yml vs application.properties
- Environment variables
- Configuration properties
- @ConfigurationProperties
- @Value
- @Conditional
- Custom auto-configuration
- Spring Boot lifecycle
- Embedded servers
- Graceful shutdown

Important annotations:

```
@SpringBootApplication  
@EnableConfigurationProperties  
@ConfigurationProperties
```

[@Profile](#)[@ConditionalOnProperty](#)

</> Java



4. Service Discovery

Concepts

- Why service discovery is required
- Dynamic service instances
- Service registration
- Service lookup
- Client-side discovery
- Server-side discovery
- Health checking
- Heartbeats

Spring Cloud Netflix Eureka

- Eureka Server
- Eureka Client
- Service registration
- Service discovery
- `defaultZone`
- Instance metadata
- Heartbeats
- Lease renewal
- Self-preservation mode
- What happens when Eureka goes down?
- Eureka caching
- Multiple Eureka servers
- High availability

Also understand alternatives:

- Kubernetes Service Discovery
- Consul
- etcd

5. API Gateway

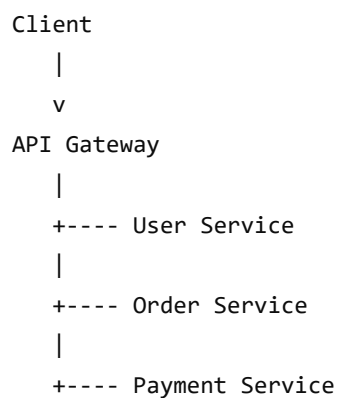
One of the most important topics.

Spring Cloud Gateway

Know:

- What is an API Gateway?
- Why not expose every microservice directly?
- Routing
- Predicates
- Filters
- Global filters
- Custom filters
- Request transformation
- Response transformation
- Path rewriting
- Authentication at gateway
- Rate limiting
- CORS
- Load balancing
- Gateway vs Load Balancer

Common architecture:



```
graph TD
    Client --> V((v))
    V --> APIGateway[API Gateway]
    APIGateway --> UserService[+---- User Service]
    APIGateway --> OrderService[+---- Order Service]
    APIGateway --> PaymentService[+---- Payment Service]
```

The diagram illustrates the common architecture of Spring Cloud Gateway. It shows a Client at the top, which connects to an API Gateway. The API Gateway then routes requests to three different microservices: User Service, Order Service, and Payment Service. The connections are represented by lines and arrows, with the API Gateway acting as a central hub.

Important concepts:

- RoutePredicateFactory
- GatewayFilter
- GlobalFilter
- RequestRateLimiter
- Redis rate limiter

6. Inter-Service Communication

Synchronous Communication

RestTemplate

Know why it is now generally considered legacy for new development.

WebClient

Understand:

- Reactive and non-blocking communication
- Mono
- Flux
- Timeout
- Retry
- Error handling

OpenFeign

Very important.

Topics:

- Declarative REST clients
- Feign client configuration
- Interceptors
- Error decoder
- Retry
- Timeout
- Service discovery integration

Example:

```
@FeignClient(name = "order-service")
public interface OrderClient {

    @GetMapping("/orders/{id}")
    Order getOrder(@PathVariable Long id);
}
```

Also know:

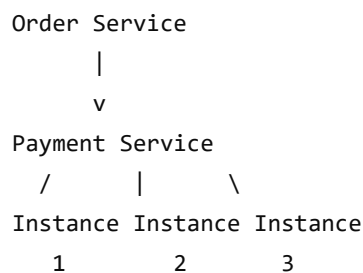
- WebClient vs Feign
- Blocking vs non-blocking communication
- When to use synchronous communication

7. Load Balancing

Topics:

- Why load balancing is needed
- Client-side load balancing
- Server-side load balancing
- Round Robin
- Weighted load balancing
- Least connections
- Spring Cloud LoadBalancer
- Load balancing with service discovery
- API Gateway load balancing
- Kubernetes load balancing

Example:



8. Centralized Configuration

Spring Cloud Config

Must know:

- Why centralized configuration
- Config Server
- Config Client
- Git-backed configuration
- Environment-specific configuration
- Profile-specific configuration
- Configuration refresh
- Encryption/decryption
- Fail-fast configuration

Important:

Config Repository

|

v

Config Server

|

+---+---+

|

User

Service

|

Order

Service



Also understand:

- Spring Cloud Bus
- Dynamic configuration refresh
- @RefreshScope

Alternatives:

- Kubernetes ConfigMaps
- Secrets
- HashiCorp Vault
- AWS Parameter Store

9. Resilience and Fault Tolerance

This is **extremely** important for 7–8 years.

You should understand:

- Why distributed systems fail
- Cascading failures
- Partial failures
- Latency
- Timeouts
- Retry
- Circuit breaker
- Bulkhead
- Rate limiting
- Fallback
- Backpressure

Resilience4j

Know:

Circuit Breaker

States:

CLOSED

|

Failure threshold exceeded

|

v

OPEN

|

Wait duration

|

v

HALF_OPEN



Topics:

- Failure rate threshold
- Sliding window
- Minimum number of calls
- Wait duration
- Half-open state

Retry

- Retry strategies
- Exponential backoff
- Jitter
- When retry is dangerous
- Retry storms

Bulkhead

- Thread pool bulkhead
- Semaphore bulkhead
- Resource isolation

Rate Limiter

- Limiting requests
- Protecting downstream services

Time Limiter

- Preventing long-running calls

Fallback

- Graceful degradation

You should know **when not to retry**.

For example:

GET → retry may be safe

POST → retry can create duplicate data



Unless the operation is idempotent.

10. Distributed Data Management

One of the biggest differences between a junior and senior microservices developer.

Topics:

- Database per service
- Shared database anti-pattern
- Data consistency
- Eventual consistency
- Strong consistency
- Distributed transactions
- Why 2-phase commit is usually avoided
- CAP theorem
- BASE vs ACID

11. Saga Pattern

Must know for senior interviews.

Types:

Choreography

```
Order Created
  |
  v
Payment Service
  |
  v
Inventory Service
  |
```



v
Shipping Service

Services communicate through events.

Orchestration

Saga Orchestrator
/ | \
v v v
Payment Inventory Shipping



Know:

- Compensating transactions
- Failure handling
- Saga state
- Idempotency
- Duplicate events
- Retry handling

12. Transactional Outbox Pattern

Very important.

Problem:

Save Order
+
Publish Event



What if:

Order saved successfully

BUT

Kafka publish failed?



This creates inconsistency.

Transactional Outbox solves this.

Topics:

- Outbox table
 - Atomic database transaction
 - Event publisher
 - Polling publisher
 - CDC
 - Debezium
 - Exactly-once myth
 - At-least-once delivery
 - Idempotent consumers
-

13. Event-Driven Architecture

Know:

- Events vs commands
- Event producers
- Event consumers
- Event brokers
- Event schemas
- Event versioning
- Event ordering
- Duplicate messages
- Dead Letter Queue
- Replay
- Idempotency

Technologies:

- Apache Kafka
- RabbitMQ
- AWS SQS/SNS
- ActiveMQ

For your experience level, **Kafka basics + practical architecture knowledge** is highly valuable.

14. Apache Kafka for Microservices

You should know:

- Broker
- Topic

- Partition
- Offset
- Consumer
- Consumer group
- Producer
- Replication factor
- Leader and follower
- Ordering
- Consumer rebalance
- Offset commit
- Auto commit vs manual commit
- At-most-once
- At-least-once
- Exactly-once concepts
- Idempotent producer
- Dead Letter Topic
- Retry topics
- Schema Registry
- Avro / JSON / Protobuf

Spring:

</> Java



Also know:

- Consumer concurrency
- Error handling
- Retries
- DLT
- Kafka transactions

15. Distributed Transactions and Consistency

Topics:

- Local transaction
- Distributed transaction
- Two-phase commit
- Why 2PC is problematic

- Saga
- Eventual consistency
- Compensating transactions
- Idempotent operations
- Exactly-once processing limitations

Important interview question:

How would you maintain consistency between Order, Payment, and Inventory services?

You should answer using **Saga + Outbox + Idempotency**.

16. Idempotency

Extremely important.

Understand:

Client sends payment request



Network timeout occurs

Client retries

Payment request executes twice ❌

Solution:

Idempotency-Key



Topics:

- Idempotent API design
 - Idempotency keys
 - Database unique constraints
 - Duplicate event processing
 - Idempotent consumers
 - Retry safety
-

17. API Design

Know:

- REST principles

- Resource naming
- HTTP methods
- HTTP status codes
- Pagination
- Sorting
- Filtering
- Versioning
- HATEOAS basics
- Idempotency
- Request validation
- Error handling
- Problem Details / RFC-style error responses
- Backward compatibility

Also understand:

GET	→ Idempotent
PUT	→ Idempotent
DELETE	→ Usually idempotent
POST	→ Usually not idempotent



18. Security in Microservices

Very important.

Topics:

- Authentication
- Authorization
- JWT
- OAuth2
- OpenID Connect
- Access token
- Refresh token
- API Gateway authentication
- Token propagation
- Service-to-service authentication
- Role-based authorization
- Scope-based authorization
- Spring Security
- Resource Server

- OAuth2 Client

Architecture:

```
Client
|
v
Identity Provider
|
JWT Token
|
v
API Gateway
|
+---- Order Service
|
+---- Payment Service
```



Know tools/concepts such as:

- Keycloak
- Okta
- Auth0
- OAuth2 Authorization Server

Also:

- mTLS
- Secret management
- Credential rotation

19. Observability

A senior developer should know the difference between:

Logging

What happened?



Metrics

How frequently is it happening?



Tracing

Where did the request travel?



Centralized Logging

Topics:

- ELK Stack
- Elasticsearch
- Logstash
- Kibana
- Fluentd / Fluent Bit
- Structured logging
- JSON logs
- Correlation ID

Distributed Tracing

Topics:

- Trace ID
- Span ID
- Parent/child spans
- Trace propagation

Tools:

- OpenTelemetry
- Jaeger
- Zipkin

Metrics

Tools:

- Micrometer
- Prometheus
- Grafana

Important metrics:

- Request rate
- Error rate
- Latency

- CPU
- Memory
- JVM metrics
- Thread pool usage
- Database connections

Golden signals:

Latency
Traffic
Errors
Saturation



20. Spring Boot Actuator

Must know well.

Endpoints:

/actuator/health
/actuator/info
/actuator/metrics
/actuator/prometheus



Topics:

- Liveness probes
- Readiness probes
- Custom health indicators
- Custom metrics
- Securing actuator endpoints

21. Distributed Logging and Correlation

Understand this flow:

Request
|
v
Gateway
Trace ID = abc123
|
v



```
Order Service
Trace ID = abc123
|
v
Payment Service
Trace ID = abc123
```

When production fails, you should be able to trace the complete request.

Topics:

- Correlation ID
 - MDC
 - Trace propagation
 - Structured logging
-

22. Docker

For 7–8 years, know more than basic commands.

Topics:

- Docker image
- Container
- Dockerfile
- Layers
- Image optimization
- Multi-stage builds
- Docker Compose
- Volumes
- Networks
- Environment variables
- Container health checks
- Container security

Java-specific:

- JVM memory inside containers
 - Container resource limits
 - Optimizing Docker images
 - Jib
 - Buildpacks
-

23. Kubernetes

Very important for modern microservices.

Know:

- Cluster
- Node
- Pod
- Deployment
- ReplicaSet
- Service
- ConfigMap
- Secret
- Namespace
- Ingress

Scaling:

- Horizontal Pod Autoscaler
- Vertical Pod Autoscaler concepts
- Cluster Autoscaler concepts

Health:

Liveness Probe
Readiness Probe
Startup Probe



Networking:

- Service discovery
- ClusterIP
- NodePort
- LoadBalancer
- Ingress

Deployment strategies:

- Rolling deployment
- Blue-green deployment
- Canary deployment

24. Service Mesh

Good advanced topic.

Know the purpose of:

- Istio
- Linkerd

Concepts:

- Sidecar proxy
- Traffic management
- mTLS
- Observability
- Retry
- Circuit breaking
- Canary releases

You don't necessarily need deep production expertise, but you should understand **why a service mesh exists**.

25. CI/CD for Microservices

Topics:

- CI vs CD
- Build pipeline
- Unit tests
- Integration tests
- Code quality checks
- Security scanning
- Docker image building
- Image registry
- Deployment pipeline
- Rollback

Tools you may encounter:

- [Jenkins ↗](#)
- [GitHub Actions ↗](#)
- [GitLab CI/CD ↗](#)
- [Argo CD ↗](#)

Also understand:

- GitOps
- Infrastructure as Code
- Helm
- Terraform basics

- Rollback strategies
-

26. Testing Microservices

A senior developer should know multiple levels.

Unit Testing

- JUnit
- Mockito

Integration Testing

- Spring Boot Test
- Testcontainers

API Testing

- MockMvc
- WebTestClient
- REST Assured

Contract Testing

Very important.

- Consumer-driven contracts
- [Spring Cloud Contract ↗](#)
- Pact concepts

End-to-End Testing

- Testing complete flows

Know the testing pyramid.

27. Caching

Topics:

- Local cache
- Distributed cache
- Redis
- Cache-aside pattern
- Write-through
- Write-behind

- TTL
- Cache invalidation
- Cache stampede

Spring:

@Cacheable

@CachePut

</> Java



Important:

How do you keep cache and database consistent?

28. Rate Limiting

Know algorithms:

- Fixed window
- Sliding window
- Token bucket
- Leaky bucket

Implementation locations:

Client

|

API Gateway ← Common place

|

Microservice



Tools:

- Redis
- Spring Cloud Gateway
- Bucket4j

29. Async Processing

Topics:

- @Async
- CompletableFuture

- Message queues
- Kafka
- RabbitMQ
- Background workers

Understand:

- Thread pools
 - Backpressure
 - Async error handling
 - Context propagation
-

30. Reactive Microservices

You don't need to be an expert, but should understand:

- Reactive programming
- Reactive Streams
- Publisher
- Subscriber
- Backpressure
- Mono
- Flux
- WebFlux
- WebClient
- Blocking vs non-blocking

Important question:

When should you use Spring MVC vs WebFlux?

31. Performance and Scalability

Topics:

- Horizontal scaling
- Vertical scaling
- Stateless services
- Connection pooling
- Thread pools
- Database bottlenecks
- Caching

- Async processing
- Load testing
- JVM tuning basics

Tools:

- JMeter
- Gatling
- k6

Metrics:

- P50
- P95
- P99 latency
- Throughput
- RPS
- Error rate

32. Deployment Strategies

Know:

Rolling Deployment

```
V1 V1 V1
↓
V2 V1 V1
↓
V2 V2 V1
↓
V2 V2 V2
```



Blue-Green

```
Blue → Current
Green → New

Switch traffic
```



Canary

```
90% → Old version
10% → New version
```



Also:

- Feature flags
 - Rollbacks
 - Backward compatibility
 - Database migration strategies
-

33. Microservices Design Patterns

You should know these patterns:

Core

- API Gateway
- Service Discovery
- Database per Service
- Saga
- CQRS
- Event Sourcing
- Strangler Fig
- Sidecar
- Ambassador
- Anti-Corruption Layer
- Backend for Frontend

Reliability

- Circuit Breaker
- Retry
- Bulkhead
- Timeout
- Rate Limiter
- Health Check

Data

- Transactional Outbox
- CQRS
- Event Sourcing
- Materialized View
- API Composition

Deployment

- Blue-Green

- Canary
 - Rolling Deployment
-

34. CQRS

Topics:

- Command model
 - Query model
 - Separate read/write databases
 - Event-driven synchronization
 - Eventual consistency
 - When CQRS is useful
 - When CQRS is overengineering
-

35. Event Sourcing

Advanced topic.

Understand:

Event 1 → Account Created
Event 2 → Money Deposited
Event 3 → Money Withdrawn



Current state is reconstructed from events.

Know:

- Event store
 - Event replay
 - Snapshots
 - Event versioning
 - Schema evolution
-

36. Cloud Knowledge

Since modern microservices are usually cloud-native, understand:

AWS

- EC2

- ECS
- EKS
- Lambda basics
- RDS
- S3
- IAM
- CloudWatch
- Load Balancer
- API Gateway
- SQS
- SNS
- Secrets Manager
- Parameter Store

Also understand:

- Cloud-native architecture
 - Autoscaling
 - High availability
 - Multi-AZ deployment
 - Disaster recovery basics
-

37. Common Microservices Problems / Anti-Patterns

This is a **very important senior-level area**.

Know:

- Distributed monolith
- God microservice
- Shared database
- Chatty communication
- Too many synchronous calls
- Cascading failures
- Missing timeouts
- Retry storms
- Distributed transactions everywhere
- Shared domain models
- Tight coupling
- Nano-services
- Lack of observability

- Missing idempotency
- Dual-write problem
- Breaking API changes

38. System Design for Microservices

At your experience level, you should be able to design systems such as:

E-commerce

API Gateway

```

|
+----- User Service
|
+----- Product Service
|
+----- Order Service
|           |
|           v
|         Kafka
|        /   \
|       v     v
|   Payment  Inventory
|
+----- Notification Service

```



You should be able to discuss:

- Service boundaries
- Database design
- Communication
- Scaling
- Caching
- Authentication
- Failure handling
- Consistency
- Monitoring
- Deployment

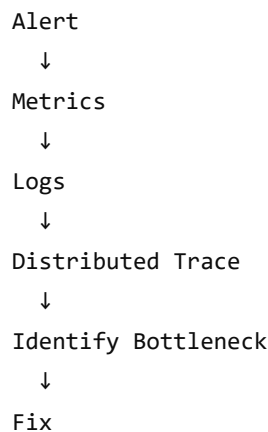
39. Production Troubleshooting

For 7–8 years, this is critical.

You should know how to investigate:

- High CPU
- High memory
- Memory leaks
- High latency
- Database connection exhaustion
- Thread pool exhaustion
- Kafka consumer lag
- Pod crashes
- Frequent restarts
- 5xx errors
- Downstream service failures

Debugging flow:



```
graph TD; Alert --> Metrics; Metrics --> Logs; Logs --> DistributedTrace[Distributed Trace]; DistributedTrace --> IdentifyBottleneck[Identify Bottleneck]; IdentifyBottleneck --> Fix;
```

40. Architecture Decision-Making

This is what separates a **senior developer** from someone who only knows **Spring annotations**.

— You should be able to answer:

✦ Upgrade



Why did you choose this architecture?

For example:

Why Kafka instead of REST?

Why Saga instead of distributed transactions?

Why Redis?

Why Kubernetes?

Why asynchronous communication?

Why separate databases?

Why API Gateway?

Why not microservices?

You should always explain **trade-offs**.

My recommended priority for you

Don't try to master all 40 sections simultaneously.

Tier 1 — Must Master

1. Microservices fundamentals
2. Spring Boot
3. Service Discovery
4. API Gateway
5. OpenFeign / WebClient
6. Load Balancing
7. Config Server
8. Resilience4j
9. Database per Service
10. Saga
11. Transactional Outbox
12. Kafka
13. Idempotency
14. API Design
15. Spring Security + JWT + OAuth2
16. Docker
17. Kubernetes
18. Actuator
19. Logging + Distributed Tracing
20. Prometheus + Grafana
21. Testing + Testcontainers
22. Caching + Redis

Tier 2 — Strong Knowledge

23. CQRS
24. Event Sourcing
25. Service Mesh
26. Spring Cloud Bus
27. Contract Testing
28. CI/CD + GitOps
29. Deployment strategies
30. Performance testing
31. Reactive programming
32. Cloud architecture

● Tier 3 — Architecture-Level Understanding

33. DDD
34. CAP theorem
35. BASE vs ACID
36. 12-Factor Apps
37. Strangler Fig
38. Anti-Corruption Layer
39. Sidecar pattern
40. Advanced distributed systems concepts

What I would expect from a strong 7–8 year candidate

You should be able to confidently design and explain:

"Design an Order Management System using Spring Boot microservices where payment, inventory, and notification are separate services. Ensure data consistency, handle failures, avoid duplicate payments, support high traffic, monitor the system, and deploy it to Kubernetes."

If you can explain that architecture using **API Gateway → service discovery → synchronous/asynchronous communication → Kafka → Saga → Outbox → idempotency → Resilience4j → Redis → observability → Docker → Kubernetes**, you will be operating at a strong **senior Java/Spring microservices level**.

Given your Java/Spring Boot background and your goal of moving into higher-paying senior roles, I would focus especially on **Kafka, Saga/Outbox, Resilience4j, security, observability, Docker/Kubernetes, and microservices system design**—these are the areas that usually differentiate a CRUD-focused Spring Boot developer from a strong senior backend engineer.

Is this conversation helpful so far?



ChatGPT can make mistakes. Check important info.